

gan

July 6, 2026

```
[ ]: import torch
import numpy as np
import cv2
import os
import random as rand
import torchvision
import pandas as pd
from tqdm import tqdm
from torch import nn, Tensor
import matplotlib.pyplot as plt
from typing import Optional
from torch.nn import functional as F
from torchvision.transforms import v2 as T
from torchvision.utils import make_grid
from torch.utils.data import Dataset, DataLoader
from math import ceil
from torchmetrics.image.fid import FrechetInceptionDistance
from torchmetrics.image.inception import InceptionScore

[ ]: class CIFAR(Dataset):
    def __init__(self, path="/scratch/s25090/archive/cifar-10/train", dataset:
        Optional[list]=None):
        super().__init__()
        self.path = path
        self.files = os.listdir(self.path) if dataset is None else dataset
        self.T = T.Compose([
            T.ToImage(),
            T.ToDtype(torch.float32, scale=True),
            T.Resize((32, 32)),
            T.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
        ])

    def __len__(self):
        return len(self.files)

    def __getitem__(self, idx):
        file = self.files[idx]
```

```

        img_path = os.path.join(self.path, file)
        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = self.T(img)
        return img

class MNISTCSV(Dataset):
    def __init__(self, csv_file):
        # 1. Read the CSV
        data = pd.read_csv(csv_file)

        # 2. Extract Labels
        self.labels = torch.tensor(data.iloc[:, 0].values, dtype=torch.long)

        # 3. Extract Images
        # We keep them as uint8 (0-255) to save memory and let transforms
        ↪ handle scaling later
        images = data.iloc[:, 1:].values.reshape(-1, 28, 28).astype('uint8')
        self.images = torch.from_numpy(images) # Shape: (N, 28, 28)

        # 4. Define Transforms (v2)
        self.transforms = T.Compose([
            T.ToImage(), # Converts to Tensor (C, H, W)
            T.Grayscale(num_output_channels=3), # <--- FORCES 3 CHANNELS (RGB)
            T.Resize((32, 32)), # Resize to 32x32
            T.ToDtype(torch.float32, scale=True), # Converts 0-255 int to 0.0-1.
            ↪ 0 float
            T.Normalize(mean=[0.5, 0.5, 0.5], # Normalize for 3 channels
                        std=[0.5, 0.5, 0.5])
        ])

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        img = self.images[idx]
        label = self.labels[idx]

        # Apply the transformation pipeline
        if self.transforms:
            img = self.transforms(img)

        return img, label

```

```

[ ]: class GenBlock(nn.Module):
    def __init__(self, in_channel, out_channel, is_final):
        super().__init__()

```

```

layers = [
    nn.Conv2d(in_channel, (out_channel+in_channel)//2, 3, 1, 1),
    nn.BatchNorm2d((out_channel+in_channel)//2),
    nn.LeakyReLU(0.2, inplace=True),

    nn.Conv2d((in_channel+out_channel)//2, out_channel, 3, 1, 1)
]

if not is_final:
    layers.append(nn.BatchNorm2d(out_channel))
    layers.append(nn.LeakyReLU(0.2, inplace=True))

layers.append(nn.UpsamplingNearest2d(scale_factor=2))
self.layer = nn.Sequential(*layers)

def forward(self, x):
    return self.layer(x)

class DisBlock(nn.Module):
    def __init__(self, in_channel, out_channel):
        super().__init__()
        self.layer = nn.Sequential(
            nn.Conv2d(in_channel, (out_channel+in_channel)//2, 3, 1, 1),
            nn.BatchNorm2d((out_channel+in_channel)//2),
            nn.LeakyReLU(0.2),
            nn.Conv2d((out_channel+in_channel)//2, out_channel, 3, 1, 1),
            nn.BatchNorm2d(out_channel),
            nn.LeakyReLU(0.2),
            nn.MaxPool2d(2, 2)
        )

    def forward(self, x):
        return self.layer(x)

class ResGenBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()

        self.conv_block = nn.Sequential(
            nn.BatchNorm2d(in_channels),
            nn.ReLU(inplace=True),
            nn.Upsample(scale_factor=2, mode='nearest'),
            nn.Conv2d(in_channels, out_channels, 3, 1, 1, bias=False),

            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, 3, 1, 1, bias=False)

```

```

    )

    self.shortcut = nn.Sequential(
        nn.Upsample(scale_factor=2, mode='nearest'),
        nn.Conv2d(in_channels, out_channels, 1, 1, 0, bias=False)
    )

    def forward(self, x):
        return self.conv_block(x) + self.shortcut(x)

```

```

[ ]: class Generator(nn.Module):
    def __init__(self, z_dim=100):
        super().__init__()
        self.initial_linear = nn.Linear(z_dim, 1024 * 4 * 4)

        self.net = nn.Sequential(
            GenBlock(1024, 512, is_final=False),
            GenBlock(512, 256, is_final=False),
            GenBlock(256, 128, is_final=False),
            GenBlock(128, 64, is_final=False),
            GenBlock(64, 64, is_final=True),
        )

        self.final_layer = nn.Sequential(
            nn.Conv2d(64, 3, kernel_size=3, padding=1),
            nn.Tanh()
        )

    def forward(self, z):
        if len(z.shape) > 2:
            z = z.view(z.size(0), -1)

        x = self.initial_linear(z)
        x = x.view(-1, 1024, 4, 4)
        x = self.net(x)
        return self.final_layer(x)

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            DisBlock(3, 32),
            DisBlock(32, 64),
            DisBlock(64, 128),
        )

        self.classifier = nn.Sequential(

```

```

        nn.Flatten(),
        nn.Linear(128 * 4 * 4, 1)
    )

    def forward(self, x):
        x = self.net(x)
        return self.classifier(x)

class ResNetGenerator(nn.Module):
    def __init__(self, z_dim=100, base_channels=256):
        super().__init__()
        self.linear = nn.Linear(z_dim, 4 * 4 * base_channels)
        self.base_channels = base_channels

        self.blocks = nn.Sequential(
            ResGenBlock(base_channels, base_channels),
            ResGenBlock(base_channels, base_channels // 2),
            ResGenBlock(base_channels // 2, base_channels // 4),
        )

        self.final_layer = nn.Sequential(
            nn.BatchNorm2d(base_channels // 4),
            nn.ReLU(inplace=True),
            nn.Conv2d(base_channels // 4, 3, 3, 1, 1),
            nn.Tanh()
        )

    def forward(self, z):
        if z.ndim > 2: z = z.view(z.size(0), -1)

        x = self.linear(z)
        x = x.view(-1, self.base_channels, 4, 4)
        x = self.blocks(x)
        return self.final_layer(x)

class GANModel(nn.Module):
    def __init__(self, z_dim=100, is_res=True):
        super().__init__()
        self.generator = Generator(z_dim) if not is_res else ResNetGenerator()
        self.discriminator = Discriminator()
        self.z_dim = z_dim
        self.criterion = nn.BCEWithLogitsLoss()

    def forward(self, z):
        return self.generator(z)

    def compute_discriminator_loss(self, real_imgs, z):

```

```

        batch_size = real_imgs.size(0)
        device = real_imgs.device

        with torch.no_grad():
            fake_imgs = self.generator(z).detach()

        real_logits = self.discriminator(real_imgs)
        fake_logits = self.discriminator(fake_imgs)

        real_labels = torch.full_like(real_logits, 0.9, device=device)
        real_loss = self.criterion(real_logits, real_labels)
        fake_labels = torch.zeros_like(fake_logits, device=device)
        fake_loss = self.criterion(fake_logits, fake_labels)

        d_loss = (real_loss + fake_loss) / 2
        return d_loss

    def compute_generator_loss(self, z):
        fake_imgs = self.generator(z)
        fake_logits = self.discriminator(fake_imgs)
        target_labels = torch.ones_like(fake_logits).to(fake_logits.device)
        g_loss = self.criterion(fake_logits, target_labels)
        return g_loss, fake_imgs

```

```

[ ]: epochs=1000
warmup_epochs = 10
dis_schedule = np.linspace(5, 1, num=warmup_epochs)
DEVICE = 'cuda:1' if torch.cuda.is_available() else 'cpu'
gan_model = GANModel()
gan_model = gan_model.to(DEVICE)
opt_gen = torch.optim.Adam(gan_model.generator.parameters(), lr=1e-4)
opt_dis = torch.optim.Adam(gan_model.discriminator.parameters(), lr=1e-4)
train_dataset = MNISTCSV("/scratch/s25090/archive/mnist/mnist_train.csv")
test_dataset = MNISTCSV("/scratch/s25090/archive/mnist/mnist_test.csv")
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)

gen_loss_list = []
dis_loss_list = []

for epoch in range(epochs):
    tot_gen_loss = 0
    tot_dis_loss = 0
    gan_model.train()
    tqdm_data = tqdm(train_loader, desc=f"Epoch-{epoch+1}/{epochs}")
    dis_steps = ceil(dis_schedule[epoch]) if epoch+1 < warmup_epochs else 1
    i=0

```

```

for real_img, _ in tqdm_data:
    i+=1
    real_img = real_img.to(DEVICE)
    bs = real_img.size(0)
    if i%dis_steps==0:
        z_dis = torch.randn(bs, 100).to(DEVICE)
        opt_dis.zero_grad()
        dis_loss = gan_model.compute_discriminator_loss(real_img, z_dis)
        dis_loss.backward()
        opt_dis.step()
    else:
        dis_loss = torch.tensor(0.0).to(DEVICE)

    for param in gan_model.discriminator.parameters():
        param.requires_grad = False

    z = torch.randn(bs, 100).to(DEVICE)
    opt_gen.zero_grad()
    gen_loss, fake_img = gan_model.compute_generator_loss(z)
    gen_loss.backward()
    opt_gen.step()

    tot_gen_loss+=gen_loss.detach().cpu().item()
    tot_dis_loss+=dis_loss.detach().cpu().item()
    tqdm_data.set_postfix({
        "GenLoss":gen_loss.detach().cpu().item(),
        "DisLoss":dis_loss.detach().cpu().item(),
        "DisSteps":dis_steps
    })

    for param in gan_model.discriminator.parameters():
        param.requires_grad = True

gen_loss_list.append(tot_gen_loss/len(train_loader))
dis_loss_list.append(tot_dis_loss*dis_steps/len(train_loader))

print(f"Generator Loss: {gen_loss_list[-1]}\nDiscriminator Loss:
↳{dis_loss_list[-1]}")

if (epoch + 1) % 10 == 0:
    torch.save(gan_model.state_dict(), f'/scratch/s25090/gan_outputs/
↳weights/Experiment_mnist/gan_epoch_{epoch+1}.pth')
    gan_model.eval()
    with torch.no_grad():
        gan_image = gan_model(z)
        comparison = torch.cat([real_img[:8], gan_image[:8]], dim=0)

```

```

        grid = make_grid(comparison.cpu(), nrow=8, padding=2,
↪normalize=True)
        plt.figure(figsize=(12, 4))
        plt.imshow(grid.permute(1, 2, 0))
        plt.axis('off')
        plt.title(f'Top: Original | Bottom: Generated Image (Epoch
↪{epoch+1})')
        plt.savefig(f"/scratch/s25090/gan_outputs/plots/Experiment_mnist/
↪Epoch-{epoch+1}.png")
        plt.show()
        plt.close()

plt.figure(figsize=(10, 5))
plt.title("Generator vs Discriminator Loss")
plt.plot(gen_loss_list, label="Generator")
plt.plot(dis_loss_list, label="Discriminator")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.savefig(f"/scratch/s25090/gan_outputs/plots/Experiment_mnist.png")
plt.legend()
plt.show()

```

```

[ ]: DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
gan_model = GANModel()
gan_model = gan_model.to(DEVICE)
WEIGHT = torch.load("gan_outputs/mnist_gan.pth")
gan_model.load_state_dict(WEIGHT)
test_loader = DataLoader(MNISTCSV("/scratch/s25090/archive/mnist/mnist_test.
↪csv"), batch_size=128, shuffle=False)

```

```

[ ]: def get_evaluation_metrics(generator, dataloader, device, num_imgs=10000):
    """
    Calculates FID and IS for a GAN generator.

    Args:
        generator: The GAN generator model.
        dataloader: DataLoader for real images (needed for FID reference).
        device: 'cuda' or 'cpu'.
        num_imgs: Number of images to generate/use for calculation.
                   (Standard for papers is 50k, but 10k is faster for debugging).

    Returns:
        fid_score (float), is_score (float)
    """
    fid = FrechetInceptionDistance(feature=2048, normalize=True).to(device)
    inception = InceptionScore(normalize=True).to(device)

```



```

generator.eval()

print(f"--- Computing Metrics (Samples: {num_imgs}) ---")

real_count = 0
for batch, _ in tqdm(dataloader, desc="Processing Real Images"):
    batch = batch.to(device)
    remaining = num_imgs - real_count
    if remaining <= 0:
        break

    if batch.shape[0] > remaining:
        batch = batch[:remaining]
    if batch.min() < 0:
        batch = (batch + 1) / 2 # Now [0, 1]

    fid.update((batch * 255).to(torch.uint8), real=True)
    real_count += batch.shape[0]

fake_count = 0
while fake_count < num_imgs:
    batch_size = min(dataloader.batch_size, num_imgs - fake_count)

    z = torch.randn(batch_size, 100).to(device)

    with torch.no_grad():
        fake_imgs = generator(z)
        fake_imgs = (fake_imgs + 1) / 2
        fake_uint8 = (fake_imgs * 255).to(torch.uint8)

    fid.update(fake_uint8, real=False)
    inception.update(fake_uint8)

    fake_count += batch_size

print("Finalizing calculations...")
fid_score = fid.compute().item()
is_score_mean, is_score_std = inception.compute()

return fid_score, is_score_mean.item()

fid, is_score = get_evaluation_metrics(
    gan_model.generator,
    test_loader,
    DEVICE,
    num_imgs=2000
)

```

```
print(f"FID: {fid:.4f} | IS: {is_score:.4f}")
```

```
[ ]:
```